ONFIT 프로젝트 구조 및 기술 스택

크롤링 기반 개인 맞춤형 패션 코디 및 스마트 구매 추천 서비스

개발 기준: 4인 팀 · 약 2~3주 / 기존 Java·Spring Boot는 제외하고 Python FastAPI로 통일

1. 확정 기술 스택

영역	기술	역할
프론트엔드	React · Tailwind CSS · Vite	화면, 스타일 진단, 코디 및 가격 시각화
백엔드	Python · FastAPI	REST API, 회원·옷장, 추천·가격 분석
데이터베이스	TiDB Cloud	사용자·상품·가격 이력·코디 데이터
크롤링·RPA	Python · Playwright · BeautifulSoup	상품 수집, 판매처 조사, 가격 추적
데이터 처리	Pandas · NumPy · RapidFuzz	상품 매칭, 가격 통계, 추천 점수
배포·운영	Render · GitHub	정적 사이트, API, 자동 실행, 협업

2. 시스템 흐름

계층	역할 및 통신
React + Tailwind (Render Static Site)	사용자 입력 및 추천·가격 조회 → FastAPI REST API 호출
FastAPI (Render Web Service)	프로필, 옷장, 코디 추천, 동일 상품 가격 비교 및 분석
TiDB Cloud	상품·판매처·가격 이력과 사용자 데이터 저장
Python RPA Worker (별도 실행)	허용된 외부 데이터 수집 → 정제 → TiDB 저장 / 수집 로그 기록

배포 설계 주의: API 서버와 장시간 실행되는 크롤링 작업은 분리한다. Render 정기 작업 기능의 요금·실행 제한과 Playwright 실행 환경은 배포 전에 검증한다.

3. 주요 기능 구성

기능	핵심 구현
체형·색상 기반 맞춤 코디	사용자 직접 입력 우선, 인바디 파일 분석은 확장 기능
나만의 옷장	찜·보유 의류 관리, 기존 의류를 중심으로 코디 생성
가격 구매 타이밍	상품 가격 이력 저장, 평균·최저가·계절별 변동 분석
동일 상품 판매처 가격 비교	브랜드·모델번호·옵션 매칭 후 판매처별 가격 비교

권장 GitHub 폴더 구조

하나의 저장소(모노레포)를 기준으로 기능을 분리하되, FastAPI 애플리케이션은 단일 서버로 운영한다.

```
ONFIT/
├── frontend/                  # 황상욱 · React + Tailwind
│   ├── public/
│   └── src/
│       ├── assets/
│       ├── components/
│       │   ├── common/
│       │   ├── outfit/
│       │   ├── wardrobe/
│       │   └── price/
│       ├── pages/
│       │   ├── HomePage.jsx
│       │   ├── StyleProfilePage.jsx
│       │   ├── OutfitPage.jsx
│       │   ├── WardrobePage.jsx
│       │   ├── PriceTimingPage.jsx
│       │   └── PriceComparePage.jsx
│       ├── services/api.js
│       ├── App.jsx
│       └── main.jsx
│
├── backend/                   # Python · FastAPI
│   ├── app/
│   │   ├── main.py
│   │   ├── core/              # 환경설정·보안
│   │   ├── db/                # TiDB 연결·ORM
│   │   ├── schemas/           # Pydantic 요청·응답
│   │   ├── api/
│   │   │   ├── auth.py
│   │   │   ├── profiles.py
│   │   │   ├── outfits.py
│   │   │   ├── wardrobe.py
│   │   │   ├── products.py
│   │   │   └── prices.py
│   │   ├── services/
│   │   │   ├── body_styling.py
│   │   │   ├── outfit_recommender.py
│   │   │   ├── price_timing.py
│   │   │   └── product_matching.py
│   │   └── crawlers/
│   │       ├── collectors/
│   │       ├── parsers/
│   │       ├── pipelines/
│   │       └── jobs/
│   ├── tests/
│   ├── alembic/
│   └── requirements.txt
│
├── database/                  # schema.sql · seed.sql
├── docs/                      # 기획 · 설계 · API · 테스트 · 진행일지
├── .github/workflows/
├── render.yaml
├── .gitignore
└── README.md
```

팀 역할 · 데이터 · 개발 일정

4. 팀원별 주요 담당 파일

이름	담당 파일·모듈	핵심 업무
송성근 (조장)	profiles.py, wardrobe.py, outfits.py, body_styling.py, outfit_recommender.py	체형·색상 코디, 옷장 추천, 프로젝트 통합
황상옥	frontend/ 전체	React 화면, Tailwind, API 연결
안종범	product_matching.py, products.py, crawlers/collectors/	동일 상품 매칭, 판매처 비교, 최초 상품 수집
이승호	price_timing.py, prices.py, crawlers/jobs/	가격 이력, 자동 재수집, 구매 시점 분석

5. TiDB 주요 데이터 테이블

테이블	용도
users · user_profiles	회원, 체형·스타일·선호 색상
products · product_offers	정규화 상품 및 판매처별 등록 정보·가격
price_histories	판매처 상품의 날짜별 가격 이력
wardrobe_items	찜하거나 보유 중인 의류
outfits · outfit_items	추천 코디와 구성 상품
crawl_logs	크롤링 실행 상태·결과·오류

6. 3주 개발 일정

주차	공동 목표	팀원별 주요 작업
1주 차	실행 환경·DB·수집 검증	React / FastAPI / TiDB 연결, 크롤링 소스 선정 및 샘플 수집
2주 차	핵심 기능 개발	코디·옷장, 상품 매칭, 가격 수집·분석, 화면 API 연동
3주 차	통합 테스트 및 배포	오류 보완, 추천 검증, Render 배포, 산출물·시연 정리

7. 우선 완료할 통합 시나리오

허용된 외부 상품 정보 수집 → TiDB 저장 → FastAPI 상품 조회 및 분석 → React 화면에 출력. 이후 사용자 취향·옷장 기반 코디와 동일 상품 가격 비교, 구매 타이밍 분석을 단계적으로 연결한다.

유의사항: 이전 ONFIT 산출물의 Spring Boot / MySQL 항목은 FastAPI / TiDB로 수정해야 한다. 크롤링 대상의 이용 조건을 확인하고, 가격 이력이 부족할 때는 구매 적기 판단을 보류한다.